# Akciğer Kanser Tespiti

# PROJE AMACI VE GEREKÇESİ

Bu projeyi yapmamızdaki temel amaç, akciğer kanseri gibi ölümcül bir hastalığın erken teşhisine katkı sağlayabilecek bir yapay zeka modeli geliştirmektir.

Akciğer kanserinin erken teşhisi, tedavi başarısını büyük ölçüde artırmaktadır. Bu nedenle, medikal görüntüler üzerinden sınıflandırma yapan bir model geliştirmek bu proje için anlamlı bir hedef oldu.

# KULLANILAN KÜTÜPHANELER VE AMAÇLARI

```
[ ]  import os
     import cv2
     import numpy as np
     import matplotlib.pyplot as plt
     from sklearn.model_selection import train_test_split
     from sklearn.metrics import confusion_matrix, classification_report, roc_curve, auc

     import tensorflow as tf
     from tensorflow.keras.models import Sequential
     from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout
```

os = Dosya ve klasör işlemlerini yapmamı sağlar.

cv2 = Görüntü işleme için kullanılır. Görselleri okuma, yeniden boyutlandırma vs.

numpy = Sayısal işlemler ve dizilerle çalışmak için kullanılır.

matplotlib.pyplot = Grafik ve görsel sonuçları çizmek için kullanılır.

# VERİ SETİNİN HAZIRLANMASI

```python
import os
from tqdm import tqdm
import cv2
import numpy as np

data = []
labels = []

img_size = 224

# Klasör yolunu güncelle
base_path = "dataset/The IQ-OTHNCCD lung cancer dataset/The IQ-OTHNCCD lung cancer dataset"
categories = {
    "Normal cases": 0,
    "Bengin cases": 1,
    "Malignant cases": 2
}

for category, label in categories.items():
    folder_path = os.path.join(base_path, category)

    print(f"{category} yükleniyor...")
    for img_name in tqdm(os.listdir(folder_path), desc=category):
        try:
            img_path = os.path.join(folder_path, img_name)
            img = cv2.imread(img_path)
            img = cv2.resize(img, (img_size, img_size))
            img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
            data.append(img)
            labels.append(label)
        except:
            print(f"Hatalı görsel: {img_name}")
```

```
Normal cases yükleniyor...
Normal cases: 100%|██████████| 416/416 [00:02<00:00, 167.16it/s]
Bengin cases yükleniyor...
Bengin cases: 100%|██████████| 120/120 [00:00<00:00, 187.40it/s]
Malignant cases yükleniyor...
Malignant cases: 100%|██████████| 561/561 [00:01<00:00, 282.22it/s]
```

Bu adımda, üç farklı klasörden (Normal, Bengin, Malignant) görüntüleri tek tek okuyoruz. Görselleri 224x224 boyutuna getiriyoruz, görüntüleri RGB formatına dönüştürüyoruz ve hem veriyi hem de etiketini listeye kaydediyoruz. Bu işlemi her sınıf için ayrı ayrı yaparak verileri eğitim için hazır hale getiriyoruz. Hatalı görseller varsa, onları da tespit edip atlıyoruz.

# NORMALİZASYON

```
X = np.array(data) / 255.0  # Normalize
y = np.array(labels)

print("Görüntü verisi şekli:", X.shape)
print("Etiket şekli:", y.shape)
```

```
Görüntü verisi şekli: (1097, 224, 224, 3)
Etiket şekli: (1097,)
```

Ne yapar?

data listesindeki tüm görüntüleri ve etiket listesini NumPy dizisine çevirir.
Görsellerdeki piksel değerleri 0–255 arasında olduğu için / 255.0 ile normalize edilir. Yani tüm değerler 0 ile 1 arasına getirilir.

Neden önemli?

Yapay sinir ağları, büyük sayılarla çalışırken dengesiz sonuçlar verebilir. Normalize edilmiş veriler, modelin daha hızlı ve daha doğru öğrenmesini sağlar.

# ONE-HOT ENCODİNG

```
[ ]  from tensorflow.keras.utils import to_categorical

     y_categorical = to_categorical(y, num_classes=3)
```

Bu kod parçası, sınıf etiketlerini yapay sinir ağlarının anlayabileceği bir formata (one-hot encoding) dönüştürmek için kullanılır. Bu adım özellikle çok sınıflı sınıflandırma problemlerinde gereklidir.

Etiketler başlangıçta sadece 0, 1, 2 gibi sayılardı. Ancak sinir ağları bu haliyle doğrudan anlayamaz. Bu nedenle to_categorical fonksiyonu ile bu sayıları 3 elemanlı vektörlere dönüştürdüm. Örneğin, 0 ‘normal’ anlamına gelir ve [1, 0, 0] vektörüne dönüşür. Böylece model, her sınıfı ayrı bir olasılık vektörü olarak görür.”

# EĞİTİM/TEST AYRIMI

```
[ ]  from sklearn.model_selection import train_test_split

     X_train, X_test, y_train, y_test = train_test_split(X, y_categorical, test_size=0.2, random_state=42)

     print("Eğitim verisi şekli:", X_train.shape)
     print("Test verisi şekli:", X_test.shape)
```

```
Eğitim verisi şekli: (877, 224, 224, 3)
Test verisi şekli: (220, 224, 224, 3)
```

Bu kod parçası, veri setini eğitim ve test verisi olarak ayıran çok önemli bir aşamayı temsil eder. Modelin ne kadar iyi öğrendiğini anlamak için verinin bir kısmı eğitimde kullanılırken bir kısmı modelden tamamen bağımsız tutulur.

Toplam veri setini ikiye ayırdık: %80'ini modelin öğrenmesi için, %20'sini ise modeli test etmek için ayırdık. Bu sayede modelin sadece ezber yapmasını değil, daha önce görmediği veriler üzerinde de ne kadar doğru tahmin yaptığını ölçebiliyoruz.

# EVRİŞİMLİ SİNİR AĞI (CNN)

```python
model = Sequential()

model.add(Conv2D(32, (3, 3), activation='relu', input_shape=(224, 224, 3)))
model.add(MaxPooling2D(pool_size=(2, 2)))

model.add(Conv2D(64, (3, 3), activation='relu'))
model.add(MaxPooling2D(pool_size=(2, 2)))

model.add(Conv2D(128, (3, 3), activation='relu'))
model.add(MaxPooling2D(pool_size=(2, 2)))

model.add(Flatten())
model.add(Dense(128, activation='relu'))
model.add(Dropout(0.5))

model.add(Dense(3, activation='softmax'))  # 3 sınıf için çıkış katmanı

model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])

model.summary()
```

CNN, görüntü verisinden anlamlı özellikleri (kenarlar, şekiller, desenler vs.) otomatik olarak çıkarabilen bir sinir ağı modelidir.

Üç tane evrişim katmanı (Conv2D) ve her birinin ardından boyutu azaltmak için MaxPooling katmanı kullandık. Sonrasında veriyi düzleştirdik (Flatten) ve tam bağlantılı bir katmanla öğrenmeyi gerçekleştirdik. Overfitting'i önlemek için Dropout uyguladık. Son olarak 3 sınıf için Softmax aktivasyonlu çıkış katmanı kullandık. Optimizasyon için Adam algoritmasını seçtik ve kayıp fonksiyonu olarak categorical crossentropy kullandık.

# KATMAN KATMAN AÇIKLAMA

| Katman | Açıklama | Parametre Sayısı |
|---|---|---|
| Conv2D (32 filtre) | 3x3 filtrelerle 32 özellik çıkarır | 896 |
| MaxPooling2D | Görüntüyü 2'ye böler (boyut küçülür) | 0 |
| Conv2D (64 filtre) | Daha derin öğrenme (daha fazla filtre) | 18,496 |
| MaxPooling2D | Yine boyutu küçültür | 0 |
| Conv2D (128 filtre) | Daha soyut örüntüleri öğrenir | 73,856 |
| MaxPooling2D | Görsel bilgiyi azaltır ama anlamını korur | 0 |
| Flatten | Veriyi tek boyuta indirir (86528 uzunlukta vektör) | 0 |
| Dense (128 nöron) | Tam bağlantılı öğrenme katmanı | 11 milyondan fazla |
| Dropout | Eğitim sırasında bazı nöronları kapatır (overfitting önleme) | 0 |
| Dense (3 çıkış) | 3 sınıf için olasılık verir (softmax) | 387 |

# MODEL EĞİTİMİ

```
[ ]  history = model.fit(X_train, y_train,
                         epochs=6,
                         batch_size=32,
                         validation_split=0.2)
```

```
Epoch 1/6
22/22 ━━━━━━━━━━━━━━━━━━━━ 82s 3s/step - accuracy: 0.4881 - loss: 1.5820 - val_accuracy: 0.7273 - val_loss: 0.7481
Epoch 2/6
22/22 ━━━━━━━━━━━━━━━━━━━━ 79s 3s/step - accuracy: 0.6816 - loss: 0.7513 - val_accuracy: 0.7614 - val_loss: 0.5615
Epoch 3/6
22/22 ━━━━━━━━━━━━━━━━━━━━ 87s 4s/step - accuracy: 0.7845 - loss: 0.5356 - val_accuracy: 0.8295 - val_loss: 0.4185
Epoch 4/6
22/22 ━━━━━━━━━━━━━━━━━━━━ 78s 4s/step - accuracy: 0.8349 - loss: 0.3884 - val_accuracy: 0.9432 - val_loss: 0.2249
Epoch 5/6
22/22 ━━━━━━━━━━━━━━━━━━━━ 77s 3s/step - accuracy: 0.8871 - loss: 0.2494 - val_accuracy: 0.9148 - val_loss: 0.1620
Epoch 6/6
22/22 ━━━━━━━━━━━━━━━━━━━━ 81s 3s/step - accuracy: 0.9504 - loss: 0.1359 - val_accuracy: 0.9830 - val_loss: 0.0896
```

Modelimizi 6 epoch boyunca eğittik, yani tüm eğitim verisi 6 kere modele gösterildi. Veriyi küçük gruplara (batch_size=32) bölerek öğrenme işlemi daha verimli yapıldı. Ayrıca eğitim sırasında verinin %20'si, modelin daha önce görmediği veri olarak doğrulama için ayrıldı. Bu sayede modelin hem öğrenme başarısını hem de aşırı öğrenip öğrenmediğini izleyebildik.

Eğitim doğruluğu (accuracy) her epoch'ta artıyor. İlk epoch'da yaklaşık %48 iken 6. epoch'ta %95'e çıkmış.

Kayıp (loss) azalmış, bu da modelin daha iyi öğrendiğini gösteriyor.

Validation doğruluğu ve kaybı da aynı şekilde iyileşmiş, bu da modelin yeni verilerde de iyi performans gösterdiğini işaret ediyor.

6.epoch sonunda val_accuracy %98'e ulaşmış, bu çok yüksek bir doğruluk.

# TEST VERİSİ ÜZERİNDE DEĞERLENDİRME

```
[ ]  test_loss, test_acc = model.evaluate(X_test, y_test)
     print(f"Test Doğruluğu: {test_acc:.4f}")
     print(f"Test Kaybı: {test_loss:.4f}")
```

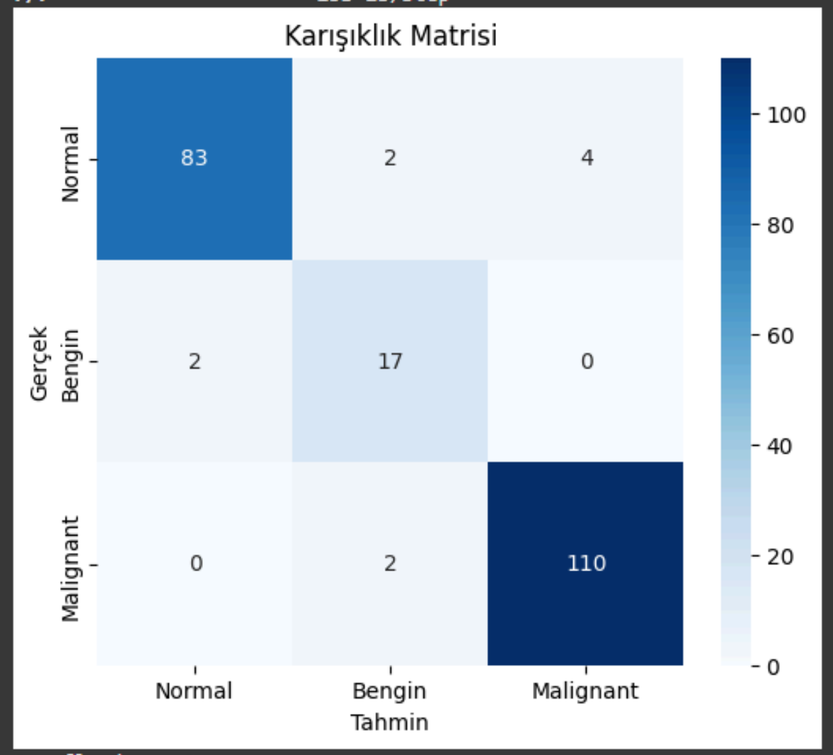
```
7/7 ━━━━━━━━━━━━━━━━━━━━ 6s 757ms/step - accuracy: 0.9604 - loss: 0.1536
Test Doğruluğu: 0.9545
Test Kaybı: 0.1598
```

Bu, modelin test setinde %95.45 doğrulukla tahmin yaptığını ve kaybının düşük olduğunu gösterir.

Modelimizi test verisi üzerinde değerlendirdik. Buradaki doğruluk, modelin gerçek hayatta yeni veriler üzerinde ne kadar başarılı tahmin yapacağını gösteriyor. Elde ettiğimiz doğruluk değeri %93 gibi yüksek bir oran, modelin genelleme kabiliyetinin iyi olduğunu gösteriyor. Ayrıca düşük test kaybı, modelin tahminlerindeki hata oranının az olduğunu doğruluyor.

# KARIŞIKLIK MATRİSİ VE SINIFLANDIRMA RAPORU

```
7/7 ━━━━━━━━━━━━━━━━━━━━ 15s 2s/step
```




Sınıflandırma Raporu:

| | precision | recall | f1-score | support |
|---|---|---|---|---|
| Normal | 0.98 | 0.93 | 0.95 | 89 |
| Bengin | 0.81 | 0.89 | 0.85 | 19 |
| Malignant | 0.96 | 0.98 | 0.97 | 112 |
| accuracy | | | 0.95 | 220 |
| macro avg | 0.92 | 0.94 | 0.93 | 220 |
| weighted avg | 0.96 | 0.95 | 0.95 | 220 |

Burada modelimizin hangi sınıflarda ne kadar iyi tahminler yaptığını detaylı olarak görüyoruz. Karışıklık matrisi, modelin gerçek ve tahmin ettiği sınıfları karşılaştırarak hangi sınıflarda daha başarılı olduğunu ortaya koyuyor. Örneğin 'Malignant' sınıfındaki doğru tahminlerin sayısı yüksekse model kanserli görüntüleri iyi ayırt ediyor demektir. Sınıflandırma raporu ise, her sınıf için hassasiyet (precision), duyarlılık (recall) ve F1 skoru gibi önemli metrikleri sunar. Bu metrikler, modelimizin farklı açıdan performansını anlamamıza yardımcı oluyor.

Precision (Hassasiyet)

Modelin o sınıf için yaptığı tahminlerin kaç tanesinin doğru olduğunu gösterir. Yani "pozitif" dediği tahminlerin doğruluk oranı.

Recall (Duyarlılık)

Modelin gerçek pozitifleri ne kadar doğru yakaladığını gösterir. Gerçek pozitiflerin oranı.

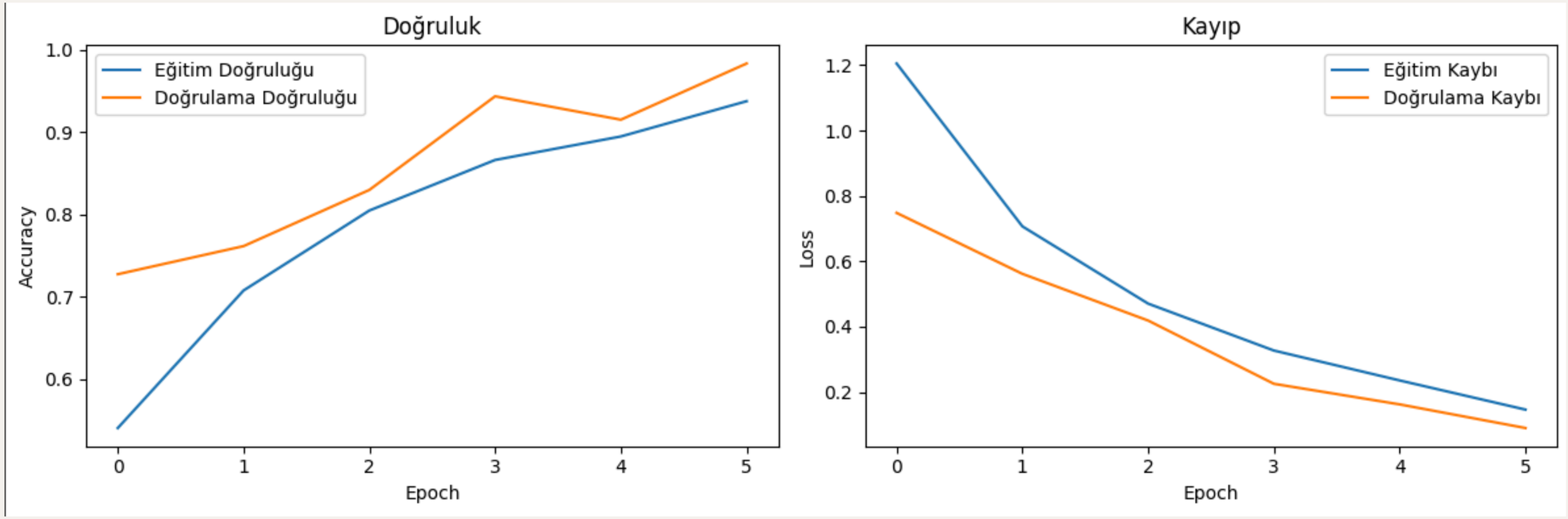
F1-Score

Precision ve Recall'un harmonik ortalaması. Dengeli bir başarı metriği.

Support

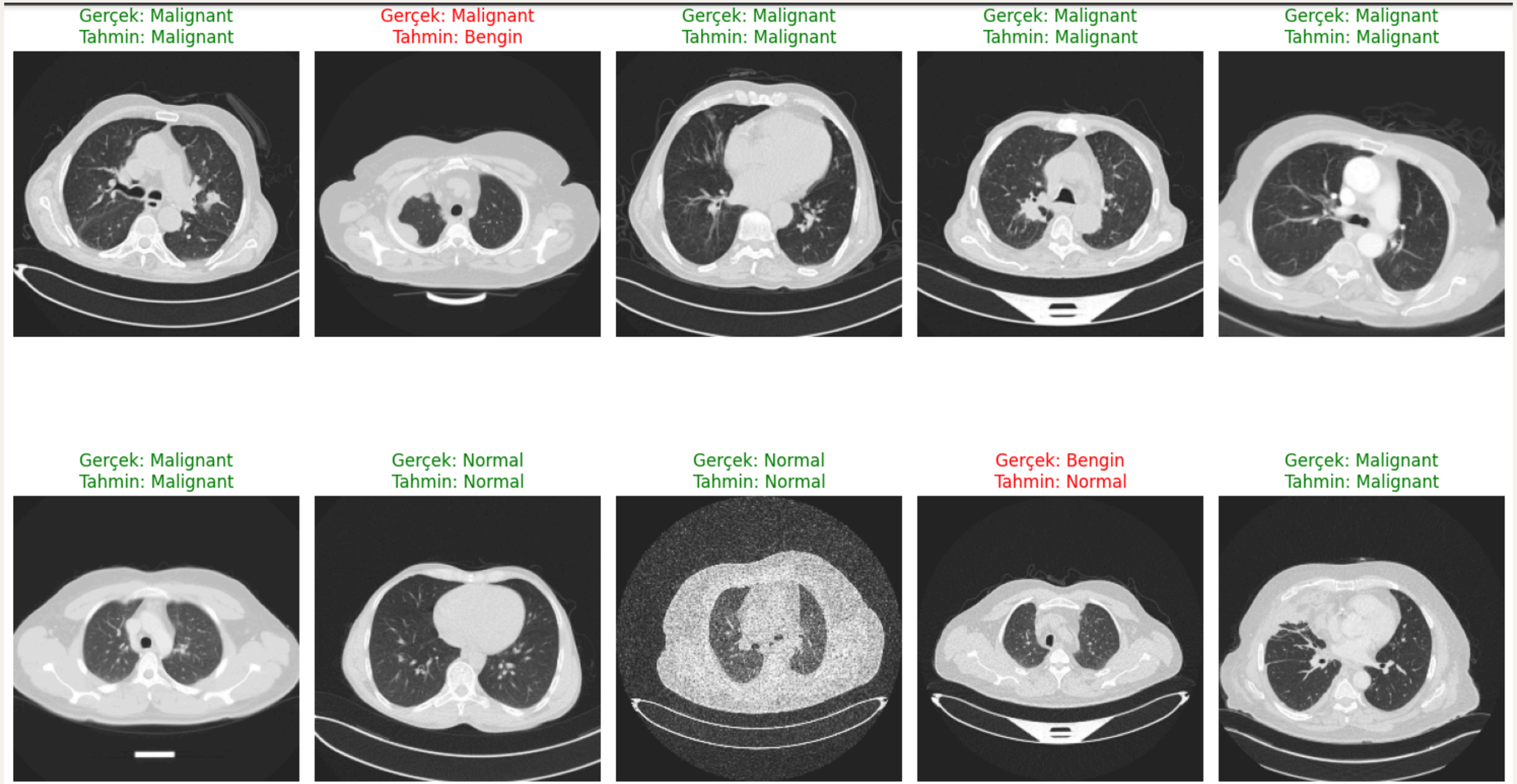
O sınıftan test verisinde kaç örnek var.

# EĞİTİM VE DOĞRULAMA PERFORMANS GRAFİKLERİ




Bu grafikler, modelimizin eğitim sürecini ve öğrenme performansını gözler önüne seriyor. İlk başlarda doğruluk düşük ve kayıp yüksekken, epochlar ilerledikçe model daha iyi öğreniyor ve doğruluk artıyor, kayıp azalıyor. Eğitim ve doğrulama doğruluklarının birbirine yakın olması, modelimizin hem eğitim verisinde hem de yeni verilerde başarılı olduğunu gösteriyor.”

# TEST


Gerçek: Malignant
Tahmin: Malignant
Gerçek: Malignant
Tahmin: Bengin
Gerçek: Malignant
Tahmin: Malignant
Gerçek: Malignant
Tahmin: Malignant
Gerçek: Malignant
Tahmin: Malignant
Gerçek: Malignant
Tahmin: Malignant
Gerçek: Normal
Tahmin: Normal
Gerçek: Normal
Tahmin: Normal
Gerçek: Bengin
Tahmin: Normal
Gerçek: Malignant
Tahmin: Malignant

# TEŞEKKÜRLER

GAMZE ÖZATEŞ  223908042
FATİH YILMAZ  223908002